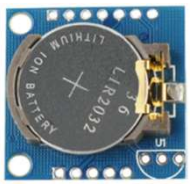
Microcontroladores

Semana 11

Por Kalun José Lau Gan

¿Preguntas previas?

- Puede explicarnos mas sobre interrupciones vectorizadas en el XC8 con el Curiosity Nano?
  - Hoy atenderemos este punto nuevamente
- ¿Cómo puedo construir un reloj con el Curiosity Nano?
  - Empleando el Timer1 ó algún módulo RTC externo (DS1307, DS3231 ó similares)







- ¿MPLAB Xpress puede correr fuera de una PC?
  - Como se evidenció la semana pasada, MPLAB Xpress al ser una plataforma que corre en un navegador web, se puede emplear cualquier máquina que tenga ello (un web browser), pudiendo ser una PC desktop corriendo Windows, Linux, MacOS, Android. Hasta desde un celular puedes desarrollar una aplicación con el Curiosity Nano!

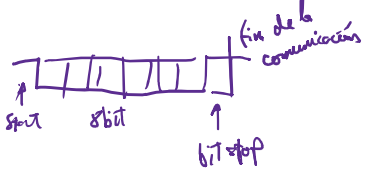
Preguntas previas

- ¿Veremos manipulación de servos?
  - Si, hoy atenderemos ese tema
- **¿Cuáles son los temas pendientes en el curso?**
  - Comunicación serial I2C, SPI (Sem 12)
  - Comunicación serial UART (Sem 13)
  - Protocolos e interfaces propietarias (sensores DHT11, LEDs Neopixel WS2812, DS18B20 1-wire), Ultrasonido HC-SR04 (Sem 14)
  - Otras interfaces y protocolos (RS485, CAN, Bluetooth, Wi-Fi, ESP-Now, MODBUS, etc) (Sem14)
  - PCB (CNC milling) y modelado 3D y manufactura aditiva (Sem 15)
  - Micros de 32bits (ESP32, Rpi-Pico, STM32, PIC32) (Sem 15)
- **¿Cuáles son las evaluaciones pendientes en el curso?**
  - Según el sílabo: LB3 (Sem12), PC2(Sem14), DD(Sem15) y TF(Sem16)

Preguntas previas

- ¿Cómo se transmiten los datos en el UART?
  - Usando el registro UxTXB (U1TXB por ejemplo)

```
U1TXB = 'H';
while (U1ERRIRbits.TXMTIF == 0);
U1TXB = 'O';
while (U1ERRIRbits.TXMTIF == 0);
U1TXB = 'L';
while (U1ERRIRbits.TXMTIF == 0);
U1TXB = 'A';
while (U1ERRIRbits.TXMTIF == 0);
```




```
unsigned int bateria = 16399;
// determinar: primero H y luego L
U1TXB = (bateria >> 8) & 0x00FF;
while (U1ERRIRbits.TXMTIF == 0);
U1TXB = bateria & 0x00FF;
while (U1ERRIRbits.TXMTIF == 0);
```

Preguntas 2024-2

- Puedo usar LiquidCrystal para el PIC18F57Q43?
  - Es una librería desarrollada para la plataforma Arduino, es incompatible con lo que se desarrolla en XC8, se tiene que reinterpretar la librería para el PIC18F57Q43.
  - Revisar repositorio del profesor Kalun para mayor información de librerías y casos
- ¿A qué se refiere con la dirección esclavo en el I2C?
  - Se refiere a la dirección única del dispositivo I2C a conectarse, si se usa varios dispositivos se debe de revisar que no haya conflicto de direcciones

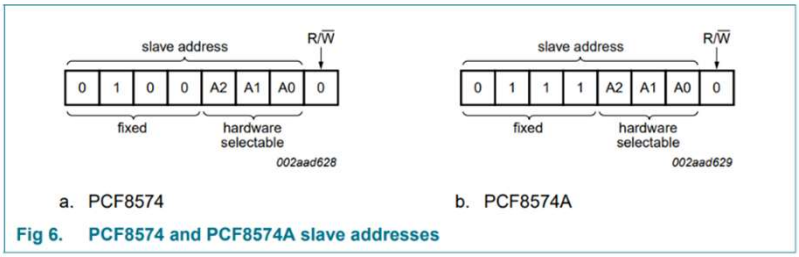


Fig 6. PCF8574 and PCF8574A slave addresses



Preguntas 2025-2

- ¿Algún programa simple de aprender para desarrollar PCBs?
  - KiCAD (gratuito)
  - EasyEDA (gratuito y online)
  - Autodesk Eagle (gratis previo registro con cuenta de correo académico)
  - Autodesk Fusion (gratis previo registro con cuenta de correo académico)
  - FluxAI (de pago)
  - Proteus ARES (de pago)
  - Altium Designer (de pago)
- ¿Cuál es la temperatura recomendada para soldar con cautín?
  - 340°C y 360° para componentes thru-hole, 320°C para SMD, siempre que se trabaje con soldadura 60/40. Se recomienda utilizar FLUX

Preguntas 2026-20

- Si necesito conectar un LCD gráfico al PIC18F47Q10. ¿Cómo puedo usarlo?
  - En el LCD HD44780 usábamos la opción de caracteres personalizados para mostrar en la pantalla símbolos que no se encuentran en el ROM de caracteres, tenemos la limitación se solo poder visualizar 8 caracteres personalizados. Esto limita la visualización de gráficos ya que la pantalla está concebida para mostrar solo caracteres.
  - Se tienen diversos modelos para display gráfico:



Controlador KS0108

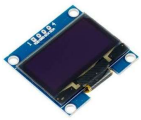


Controlador T6963C


Controlador SSD1306



Controlador SSD1309


Controlador SH1106

Preguntas 2026-20

- ¿Va a proporcionar mas librerías para usar en los proyectos del curso?
  - No, no se tienen librerías para todos los casos y/o periféricos, se deberá de hacer una pesquisa para ver si es que hay disponible en ootras plataformas, o en última instancia, desarrollar su propia librería basándose en la hoja técnica del periférico.
  - Pueden hacer uso de la IA (ChatGPT, Gemini, Claude) para generar las librerías que requieren.
- ¿Es necesario presentar el TF en una PCB?
  - Si, es MANDATORY.
- Para el LB3 del día jueves que declararon feriado hasta 1PM. ¿Cómo será la evaluación?
  - Se esta planificando para que sea a modalidad virtual con entrega en una actividad en el AV, mas detalles en el anuncio que se va a enviar culminando la sesión de hoy.

Agenda:

- Resumen de interrupciones vectorizadas
- El Timer1
- Aplicación de reloj en tiempo real con Timer1
- El servomecanismo
- Aplicación de replicación de movimiento entre un potenciómetro y un servomecanismo

Interrupciones en el PIC18F47Q10 a detalle

- Revisar en el datasheet el capítulo relacionado a las interrupciones
- Plantilla de funciones de interrupciones en XC8 en alto nivel:

```
//--------- un sola fuente de interrupcion
void __interrupt() INT_ISR(void){
//--
}
```

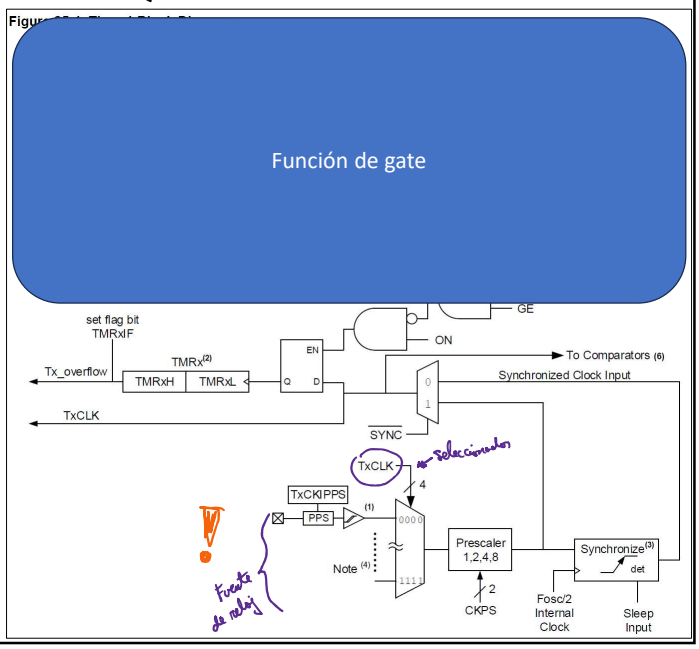
IPEN=1

```
//--------- con prioridades en modo legacy
void __interrupt(high_priority) INT_HP_ISR(void){
//--
}

void __interrupt(low_priority) INT_LP_ISR(void){
//--
}
```

El Timer 1 en el PIC18F47Q10

- Módulo temporizador de **16 bits**
- Empleado para aplicaciones de RTC (Real-time clock)
- Función de gate
- Funciona con el CCP1 (modo captura ó modo comparación)
- Múltiples fuentes de reloj
- Si se va a emplear HFINTOSC, no emplear 12MHz ni 48MHz
- El registro de la cuenta del TMR1 esta desplegado en dos registros: TMR1H:TMR1L
- Similar para Timer3 y Timer5



El Timer 1 en el PIC18F47Q10

- Selección de la fuente de reloj (registro T1CLK)
- Dos opciones para RTC:
  - Cristal de 32.768KHz (configurar como oscilador externo – en el PIC18F47Q10 es el oscilador secundario)
  - Oscilador principal del microcontrolador (uno externo ó el HFINTOSC)



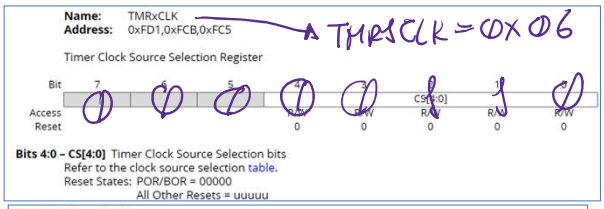
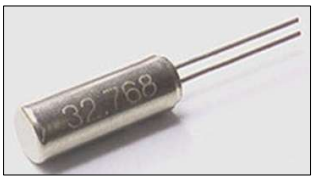


Table 20-2. Timer Clock Sources

CS	Clock Source		
	Timer1	Timer3	Timer5
11000-11111	Reserved	Reserved	Reserved
10111	CLC8_out	CLC8_out	CLC8_out
10110	CLC7_out	CLC7_out	CLC7_out
10101	CLC6_out	CLC6_out	CLC6_out
10100	CLC5_out	CLC5_out	CLC5_out
10011	CLC4_out	CLC4_out	CLC4_out
10010	CLC3_out	CLC3_out	CLC3_out
10001	CLC2_out	CLC2_out	CLC2_out
10000	CLC1_out	CLC1_out	CLC1_out
01111-01100	Reserved	Reserved	Reserved
01011	TMR5 overflow	TMR5 overflow	Reserved
01010	TMR3 overflow	Reserved	TMR3 overflow
01001	Reserved	TMR1 overflow	TMR1 overflow
01000	TMR0 overflow	TMR0 overflow	TMR0 overflow
00111	CLKREF	CLKREF	CLKREF
00110	SOSC	SOSC	SOSC
00101	MFINTOSC (500 kHz)	MFINTOSC (500 kHz)	MFINTOSC (500 kHz)
00100	LFINTOSC	LFINTOSC	LFINTOSC
00011	HFINTOSC	HFINTOSC	HFINTOSC
00010	Fosc	Fosc	Fosc
00001	Fosc/4	Fosc/4	Fosc/4
00000	T1CKIPPS	T3CKIPPS	T5CKIPPS

El Timer 1 en el PIC18F47Q10

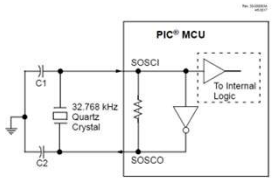
- Oscilador secundario (cap 5 – módulo de oscilador)



5.2.1.5 Secondary Oscillator

The secondary oscillator is a separate oscillator block that can be used as an alternate system clock source. The secondary oscillator is optimized for 32.768 kHz, and can be used with an external crystal oscillator connected to the SOSCI and SOSCO device pins, or an external clock source connected to the SOSCIN pin. The secondary oscillator can be selected during run-time using clock switching.

Figure 5-5. Quartz Crystal Operation (Secondary Oscillator)



Notes:

1. Quartz crystal characteristics vary according to type, package and manufacturer. The user should consult the manufacturer data sheets for specifications and recommended application.
2. Always verify oscillator performance over the V_{DD} and temperature range that is expected for the application.
3. For oscillator design assistance, reference the following Microchip Application Notes:
   - AN826, *"Crystal Oscillator Basics and Crystal Selection for PIC® and PIC® Devices"* (DS00826)
   - AN849, *"Basic PIC® Oscillator Design"* (DS00849)
   - AN943, *"Practical PIC® Oscillator Analysis and Design"* (DS00943)
   - AN949, *"Making Your Oscillator Work"* (DS00949)
   - TB097, *"Interfacing a Micro Crystal MS1V-T1K 32.768 kHz Tuning Fork Crystal to a PIC16F690/SS"* (DS91097)
   - AN1288, *"Design Practices for Low-Power External Oscillators"* (DS01288)

Registro T1CON

Name: TxCON
Address: 0xFCE,0xFC8,0xFC2

T1CON = 0x07

Timer Control Register

Bit	7	6	5	4	3	2	1	0
	0	0	CKPS[1:0]		0	SYNC	RD16	ON
Access			R/W	R/W		R/W	R/W	R/W
Reset			0	0		0	0	0

Bits 5:4 – CKPS[1:0] Timer Input Clock Prescale Select bits
Reset States: POR/BOR = 00
All Other Resets = uu

Value	Description
11	1:8 Prescale value
10	1:4 Prescale value
01	1:2 Prescale value
00	1:1 Prescale value

Bit 2 – SYNC Timer External Clock Input Synchronization Control bit
Reset States: POR/BOR = 0
All Other Resets = u

Value	Condition	Description
X	CS = F_{OSC}/4 or F_{OSC}	This bit is ignored. Timer uses the incoming clock as is.
1	Else	Do not synchronize external clock input
0	Else	Synchronize external clock input with system clock

Bit 1 – RD16 16-Bit Read/Write Mode Enable bit
Reset States: POR/BOR = 0
All Other Resets = u

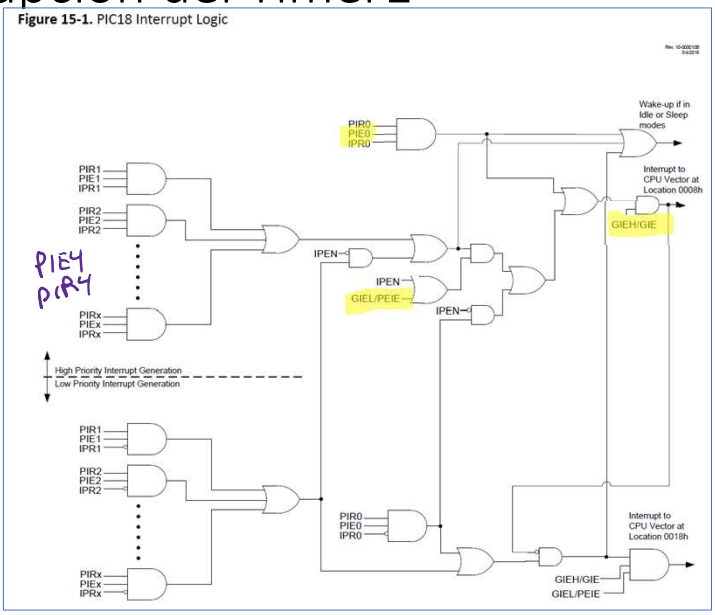
Value	Description
1	Enables register read/write of Timer in one 16-bit operation
0	Enables register read/write of Timer in two 8-bit operations

Bit 0 – ON Timer On bit
Reset States: POR/BOR = 0
All Other Resets = u

Value	Description
1	Enables Timer
0	Disables Timer

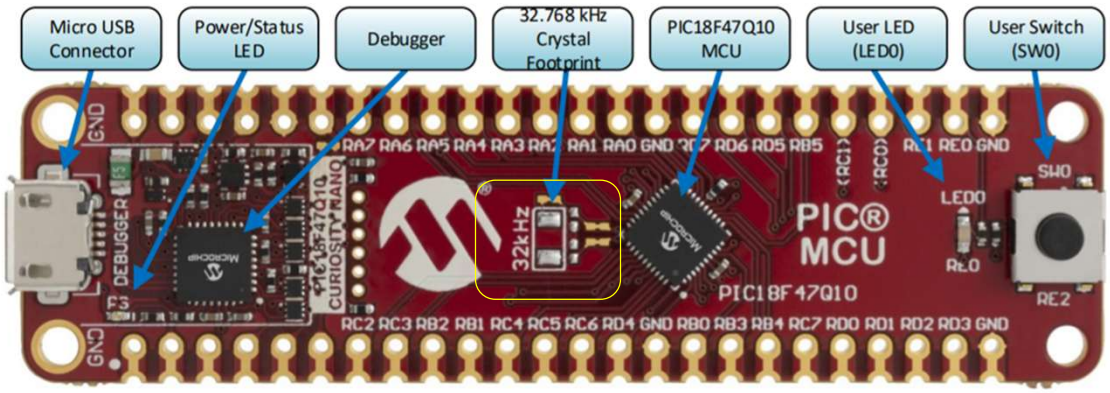
Interrupción del Timer1

Figure 15-1. PIC18 Interrupt Logic

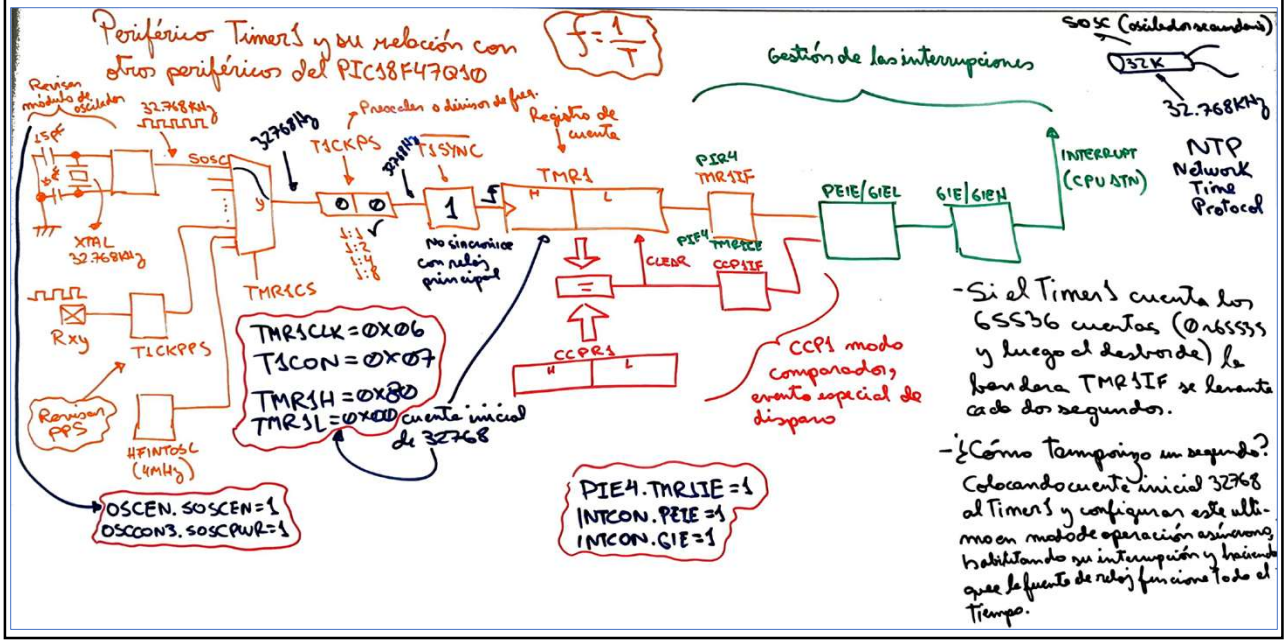


Montaje del cristal de 32.768KHz en el Curiosity Nano PIC18F47Q10:





Timer1 y periféricos relacionados con su funcionamiento



El CCPx en el PIC18F47Q10

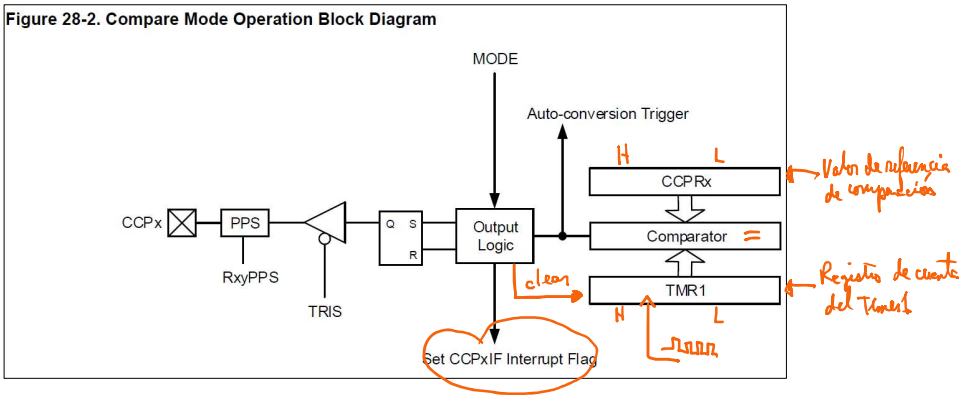
- Módulo captura/comparación/PWM (CCPx)

CCP Mode	Timer Resource
Capture	Timer1, Timer3 or Timer5
Compare	
PWM	Timer2, Timer4 or Timer6

El CCPx en el PIC18F47Q10

- El CCP1 posee tres modos de trabajo: Captura, Comparación y PWM
- CCP1 en modo comparación, evento especial de disparto se empleará para hacer la tarea de RTC junto con el Timer1
- Diagrama de bloques

Figure 28-2. Compare Mode Operation Block Diagram



El CCPx en el PIC18F47Q10

- CCP1 en modo comparación
- Información importante:

22.3.2 Timer1 Mode Resource

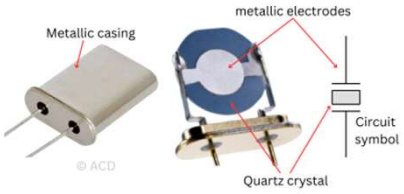
In Compare mode, Timer1 must be running in either Timer mode or Synchronized Counter mode. The compare operation may not work in Asynchronous Counter mode.

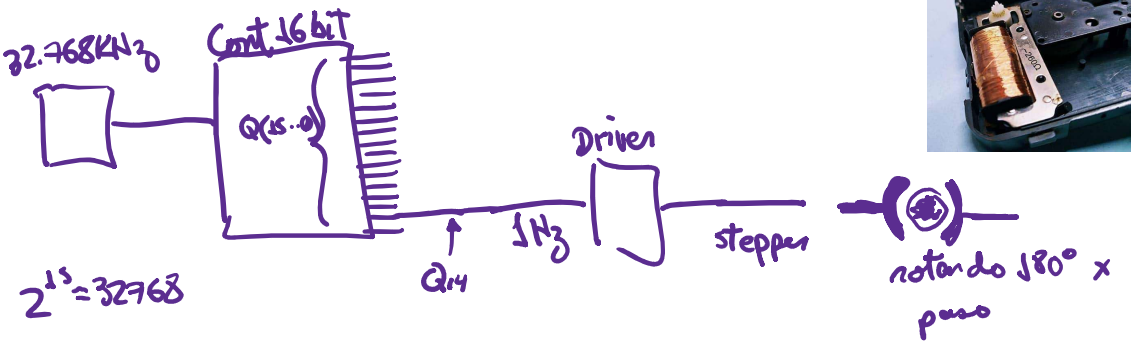
See Section "Timer1 Module with Gate Control" for more information on configuring Timer1.

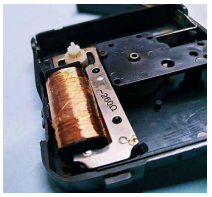
Important: Clocking Timer1 from the system clock (F_{OSC}) should not be used in Compare mode. In order for Compare mode to generate the trigger event on the CCPx pin, Timer1 must be clocked from the instruction clock ($F_{OSC}/4$) or from an external clock source.

SOSC → 32.768 kHz

¿Y por qué la fuente tiene que ser de 32.768KHz?







A revisar en la hoja técnica del PIC18F47Q10:

- Timer1:
  - Diagrama de bloques
  - Registros implicados (T1CON, etc)
- CCPx:
  - Diagrama de bloques
  - Registros implicados (CCP1CON, etc)
  - Configuración para que se active su interrupción (los registros donde están CCP1IE, CCP1IF, GIE)

Equipo para medir la precision de un reloj electrónico basado en cristal de cuarzo:



Usando el cristal de 32.768K y solo el Timer1 para aplicaciones de tiempo real

- Información de la hoja técnica del PIC18F4550 y que se ha omitido en la hoja técnica del PIC18F47Q10

12.6 Using Timer1 as a Real-Time Clock

Adding an external LP oscillator to Timer1 (such as the one described in **Section 12.3 "Timer1 Oscillator"**) gives users the option to include RTC functionality to their applications. This is accomplished with an inexpensive watch crystal to provide an accurate time base and several lines of application code to calculate the time. When operating in Sleep mode and using a battery or supercapacitor as a power source, it can completely eliminate the need for a separate RTC device and battery backup.

The application code routine, `RTCisr`, shown in Example 12-1, demonstrates a simple method to increment a counter at one-second intervals using an Interrupt Service Routine. Incrementing the TMR1 register pair to overflow triggers the interrupt and calls the routine, which increments the seconds counter by one. Additional counters for minutes and hours are incremented as the previous counter overflows.

Since the register pair is 16 bits wide, counting up to overflow the register directly from a 32.768 kHz clock would take 2 seconds. To force the overflow at the required one-second intervals, it is necessary to pre-load it. The simplest method is to set the MSb of TMR1H with a `BSF` instruction. Note that the TMR1L register is never preloaded or altered; doing so may introduce cumulative error over many cycles.

For this method to be accurate, Timer1 must operate in Asynchronous mode and the Timer1 overflow interrupt must be enabled (PIE1<0> = 1) as shown in the routine, `RTCinit`. The Timer1 oscillator must also be enabled and running at all times.

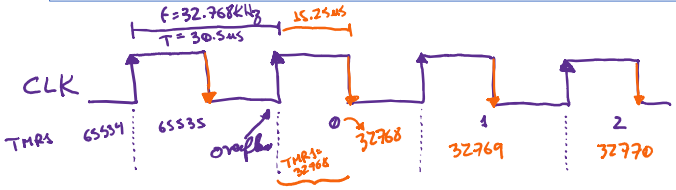
12.7 Considerations in Asynchronous Counter Mode

Following a Timer1 interrupt and an update to the TMR1 registers, the Timer1 module uses a falling edge on its clock source to trigger the next register update on the rising edge. If the update is completed after the clock input has fallen, the next rising edge will not be counted.

If the application can reliably update TMR1 before the timer input goes low, no additional action is needed. Otherwise, an adjusted update can be performed following a later Timer1 increment. This can be done by monitoring TMR1L within the interrupt routine until it increments, and then updating the TMR1H:TMR1L register pair while the clock is low, or one-half of the period of the clock source. Assuming that Timer1 is being used as a Real-Time Clock, the clock source is a 32.768 kHz crystal oscillator; in this case, one-half period of the clock is 15.25 µs.

The Real-Time Clock application code in Example 12-1 shows a typical ISR for Timer1, as well as the optional code required if the update cannot be done reliably within the required interval.

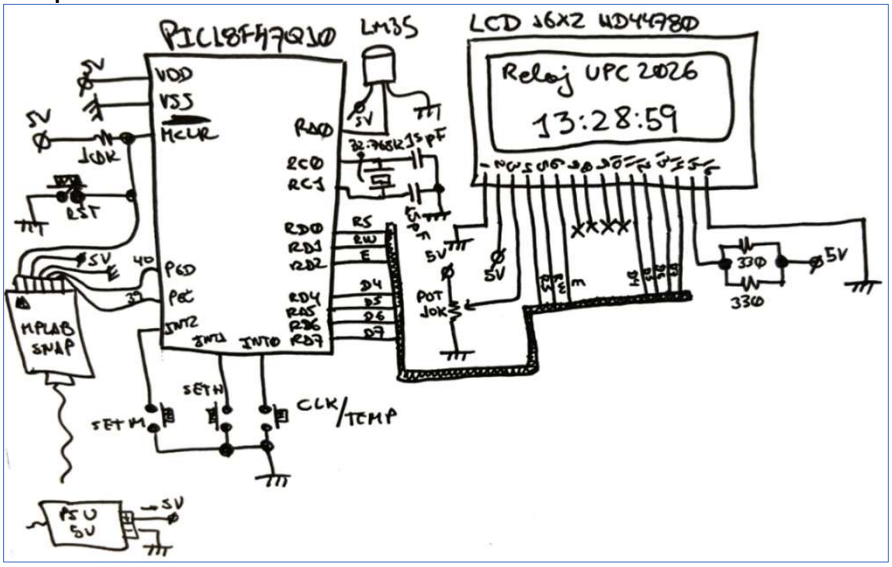




Recordar que se debe de configurar en modo asíncrono el Timer1, activar su interrupción y hacer que su fuente de reloj funcione todo el tiempo. Cuando se desborde la cuenta se tiene una ventana de 15.25us para que puedas precargar la cuenta inicial

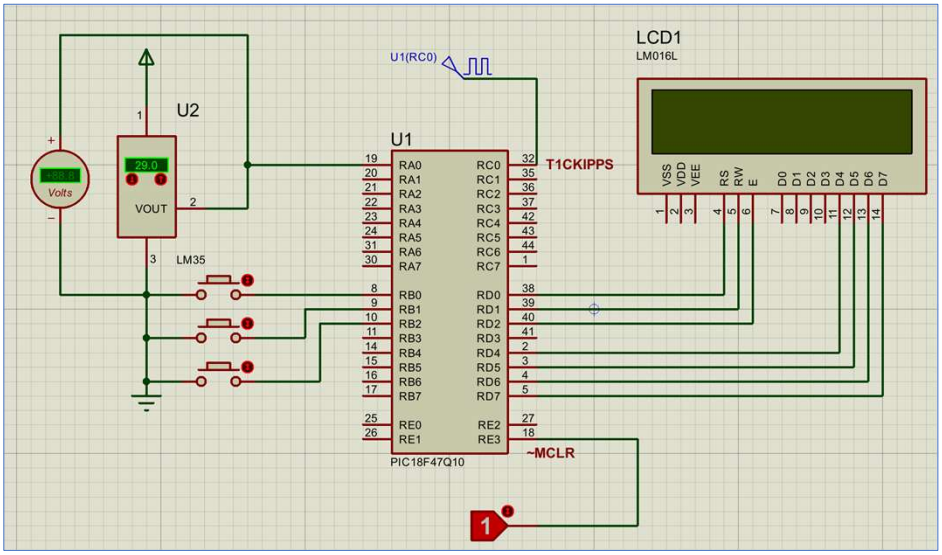
Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K

- Implementar el siguiente circuito, los capacitores en el cristal de 32.768KHz pueden omitirse

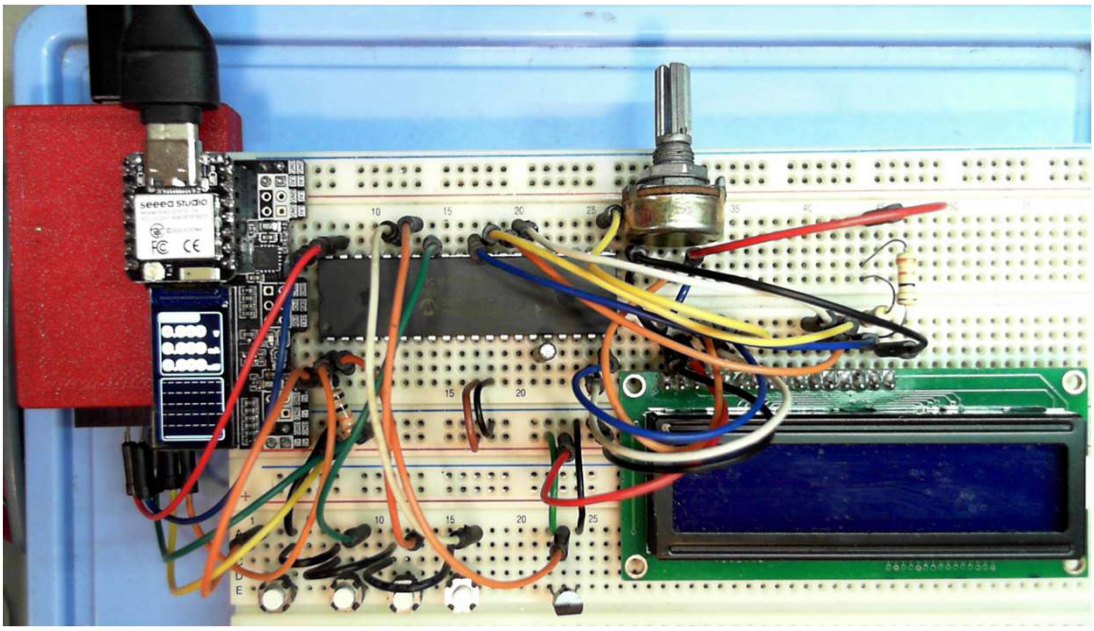


Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K

- De usar el Proteus para una simulación, éste no posee modelos de cristal de 32.768KHz para las pruebas, en lugar de ello se usa un DCLOCK a 32.768KHz y en las configuraciones de fuente de reloj del Timer1 configurar a fuente de reloj externa T1CKIPPS

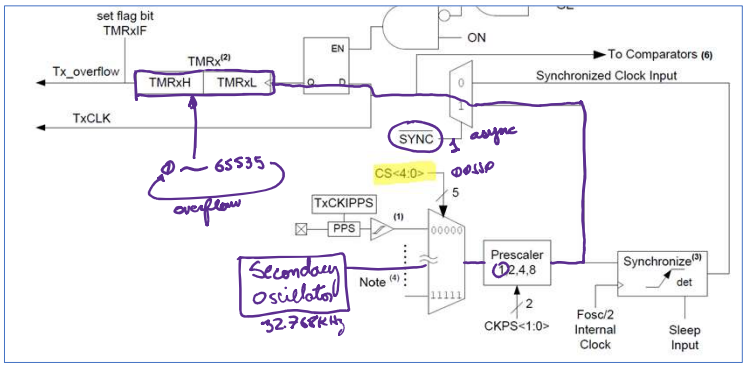


Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K



Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K

- Configuración del Timer1 (modo async, con interrupciones y reloj funcionando todo el tiempo)



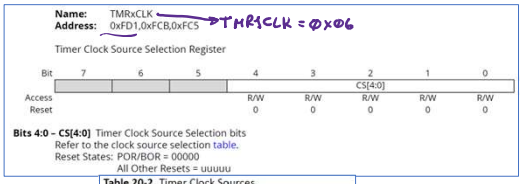
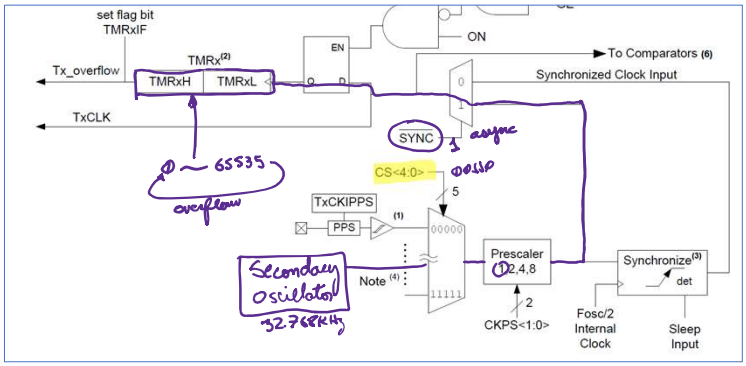


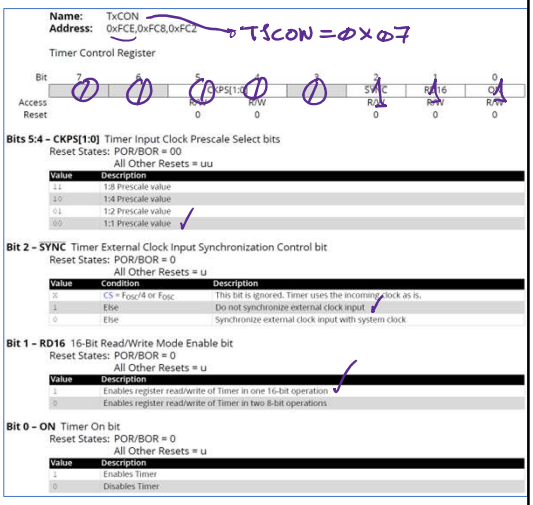
Table 20-2. Timer Clock Sources

CS	Timer1
11000-11111	Reserved
10111	CLC8_out
10110	CLC7_out
10101	CLC6_out
10100	CLC5_out
10011	CLC4_out
10010	CLC3_out
10001	CLC2_out
10000	CLC1_out
01111-01100	Reserved
01011	TMR5 overflow
01010	TMR3 overflow
01001	Reserved
01000	TMR0 overflow
00111	CLKREF
00110	SOSC
00101	MFINTOSC (500 kHz)
00100	LFINTOSC
00011	HFINTOSC
00010	F_{OSC}
00001	$F_{OSC}/4$
00000	T1CKIPPS

Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K

- Configuración del Timer1 (modo async, con interrupciones y reloj funcionando todo el tiempo)





Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K

```c
#include <xc.h>
#include "cabecera.h"
#include "LCD.h"
#define _XTAL_FREQ 4000000UL

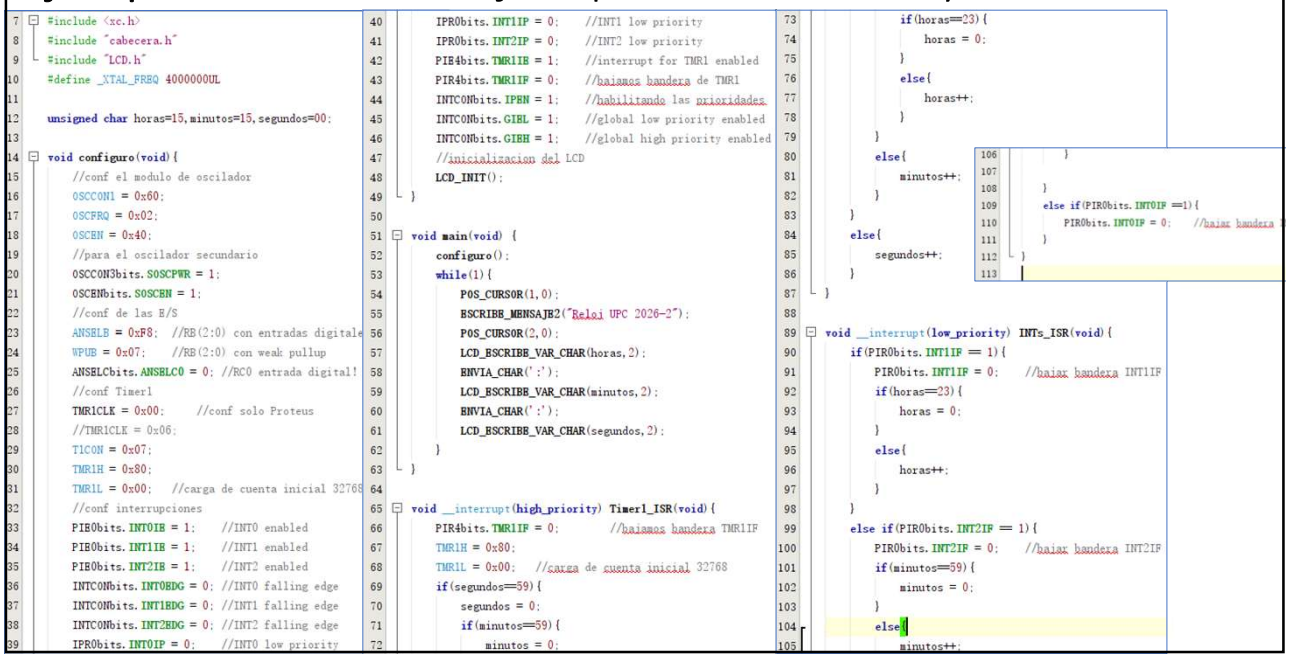
unsigned char horas=15,minutos=02,segundos=00;

void configuro(void){
    //conf el modulo de oscilador
    OSCCON1 = 0x60;
    OSCFRQ = 0x02;
    OSCEN = 0x40;
    //para el oscilador secundario
    OSCCON3bits.SOSCPWR = 1;
    OSCENbits.SOSCEN = 1;
    //conf de las E/S
    ANSELB = 0xF8;
    WPUB = 0x07;
    //conf Timer1
    T1CLK = 0x06;
    T1CON = 0x07;
    TMR1H = 0x80;
    TMR1L = 0x00;   //carga de cuenta inicial 32768
    //conf interrupciones
    PIE4bits.TMR1IE = 1;    //interrupt for TMR1 enabled
    PIR4bits.TMR1IF = 0;    //bajamos bandera de TMR1
    INTCONbits.PEIE = 1;    //pheripheral interrupt enabled
    INTCONbits.GIE = 1;     //global interrupt enabled
    //inicializacion del LCD
    LCD_INIT();
}

void main(void) {
    configuro();
    while(1){
        POS_CURSOR(1,0);
        ESCRIBE_MENSAJE2("Reloj UPC 2026-2");
        POS_CURSOR(2,0);
        LCD_ESCRIBE_VAR_CHAR(horas,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(minutos,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(segundos,2);
    }
}

void __interrupt() Timer1_ISR(void){
    PIR4bits.TMR1IF = 0;
    TMR1H = 0x80;
    TMR1L = 0x00;   //carga de cuenta inicial 32768
    if(segundos==59){
        segundos = 0;
        if(minutos==59){
            minutos = 0;
            if(horas==23){
                horas = 0;
            }
            else{
                horas++;
            }
        }
        else{
            minutos++;
        }
    }
    else{
        segundos++;
    }
}
```

Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K



```c
#include <xc.h>
#include "cabecera.h"
#include "LCD.h"
#define _XTAL_FREQ 4000000UL

unsigned char horas=15,minutos=15,segundos=00;

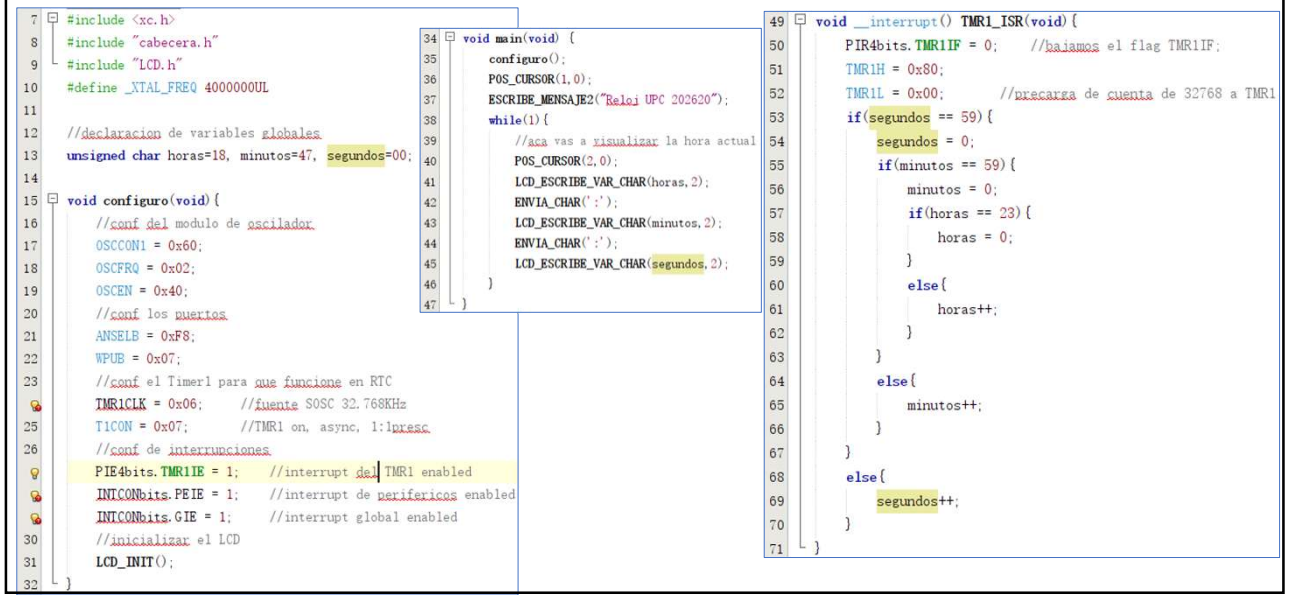
void configuro(void){
    //conf el modulo de oscilador
    OSCCON1 = 0x60;
    OSCFRQ = 0x02;
    OSCEN = 0x40;
    //para el oscilador secundario
    OSCCON3bits.SOSCPWR = 1;
    OSCENbits.SOSCEN = 1;
    //conf de las E/S
    ANSELB = 0xF8;  //RB(2:0) con entradas digitale
    WPUB = 0x07;    //RB(2:0) con weak pullup
    ANSELCbits.ANSELC0 = 0; //RC0 entrada digital!
    //conf Timer1
    TMR1CLK = 0x00;     //conf solo Proteus
    //TMR1CLK = 0x06;
    T1CON = 0x07;
    TMR1H = 0x80;
    TMR1L = 0x00;   //carga de cuenta inicial 32768
    //conf interrupciones
    PIE0bits.INT0IE = 1;    //INT0 enabled
    PIE0bits.INT1IE = 1;    //INT1 enabled
    PIE0bits.INT2IE = 1;    //INT2 enabled
    INTCON0bits.INT0EDG = 0; //INT0 falling edge
    INTCON0bits.INT1EDG = 0; //INT1 falling edge
    INTCON0bits.INT2EDG = 0; //INT2 falling edge
    IPR0bits.INT0IP = 0;    //INT0 low priority
    IPR0bits.INT1IP = 0;    //INT1 low priority
    IPR0bits.INT2IP = 0;    //INT2 low priority
    PIE4bits.TMR1IE = 1;    //interrupt for TMR1 enabled
    PIR4bits.TMR1IF = 0;    //bajamos bandera de TMR1
    INTCON0bits.IPEN = 1;   //habilitando las prioridades
    INTCON0bits.GIEL = 1;   //global low priority enabled
    INTCON0bits.GIEH = 1;   //global high priority enabled
    //inicializacion del LCD
    LCD_INIT();
}

void main(void) {
    configuro();
    while(1){
        POS_CURSOR(1,0);
        ESCRIBE_MENSAJE2("Reloj UPC 2026-2");
        POS_CURSOR(2,0);
        LCD_ESCRIBE_VAR_CHAR(horas,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(minutos,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(segundos,2);
    }
}

void __interrupt(high_priority) Timer1_ISR(void){
    PIR4bits.TMR1IF = 0;        //bajamos bandera TMR1IF
    TMR1H = 0x80;
    TMR1L = 0x00;   //carga de cuenta inicial 32768
    if(segundos==59){
        segundos = 0;
        if(minutos==59){
            minutos = 0;
            if(horas==23){
                horas = 0;
            }
            else{
                horas++;
            }
        }
        else{
            minutos++;
        }
    }
    else{
        segundos++;
    }
}

void __interrupt(low_priority) INTs_ISR(void){
    if(PIR0bits.INT1IF == 1){
        PIR0bits.INT1IF = 0;    //bajar bandera INT1IF
        if(horas==23){
            horas = 0;
        }
        else{
            horas++;
        }
    }
    else if(PIR0bits.INT2IF == 1){
        PIR0bits.INT2IF = 0;    //bajar bandera INT2IF
        if(minutos==59){
            minutos = 0;
        }
        else{
            minutos++;
        }
    }
    else if(PIR0bits.INT0IF ==1){
        PIR0bits.INT0IF = 0;    //bajar bandera
    }
}
```

Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K



```c
#include <xc.h>
#include "cabecera.h"
#include "LCD.h"
#define _XTAL_FREQ 4000000UL

//declaracion de variables globales
unsigned char horas=18, minutos=47, segundos=00;

void configuro(void){
    //conf del modulo de oscilador
    OSCCON1 = 0x60;
    OSCFRQ = 0x02;
    OSCEN = 0x40;
    //conf los puertos
    ANSELB = 0xF8;
    WPUB = 0x07;
    //conf el Timer1 para que funcione en RTC
    TMR1CLK = 0x06;     //fuente SOSC 32.768KHz
    T1CON = 0x07;       //TMR1 on, async, 1:1presc
    //conf de interrupciones
    PIE4bits.TMR1IE = 1;    //interrupt del TMR1 enabled
    INTCONbits.PEIE = 1;    //interrupt de perifericos enabled
    INTCONbits.GIE = 1;     //interrupt global enabled
    //inicializar el LCD
    LCD_INIT();
}

void main(void) {
    configuro();
    POS_CURSOR(1,0);
    ESCRIBE_MENSAJE2("Reloj UPC 202620");
    while(1){
        //aca vas a visualizar la hora actual
        POS_CURSOR(2,0);
        LCD_ESCRIBE_VAR_CHAR(horas,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(minutos,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(segundos,2);
    }
}

void __interrupt() TMR1_ISR(void){
    PIR4bits.TMR1IF = 0;    //bajamos el flag TMR1IF;
    TMR1H = 0x80;
    TMR1L = 0x00;       //precarga de cuenta de 32768 a TMR1
    if(segundos == 59){
        segundos = 0;
        if(minutos == 59){
            minutos = 0;
            if(horas == 23){
                horas = 0;
            }
            else{
                horas++;
            }
        }
        else{
            minutos++;
        }
    }
    else{
        segundos++;
    }
}
```

Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K

```c
#include <xc.h>
#include "cabecera.h"
#include "LCD.h"
#define _XTAL_FREQ 4000000UL

//declaracion de variables globales
unsigned char horas=18, minutos=47, segundos=00;

void configuro(void){
    //conf del modulo de oscilador
    OSCCON1 = 0x60;
    OSCFRQ = 0x02;
    OSCEN = 0x40;
    //conf los puertos
    ANSELB = 0xF8;
    WPUB = 0x07;
    //conf el Timer1 para que funcione en RTC
    T1RCLK = 0x06;      //fuente SOSC 32.768KHz
    T1CON = 0x07;       //TMR1 on, async, 1:1presc
    //conf de interrupciones
    PIE0bits.INT1IE = 1;     //INT1 enabled
    PIE0bits.INT2IE = 1;     //INT2 enabled
    IPR0bits.INT1IP = 0;     //INT1 low priority
    IPR0bits.INT2IP = 0;     //INT2 low priority
    INTCON0bits.INT1EDG = 0; //INT1 falling edge
    INTCON0bits.INT2EDG = 0; //INT2 falling edge
    PIE4bits.TMR1IE = 1;     //interrupt del TMR1 enabled
    INTCON0bits.IPEN = 1;    //interrupt priority enabled
    INTCON0bits.GIEL = 1;    //global low priority enabled
    INTCON0bits.GIEH = 1;    //global high priority enabled
    //inicializar el LCD
    LCD_INIT();
}

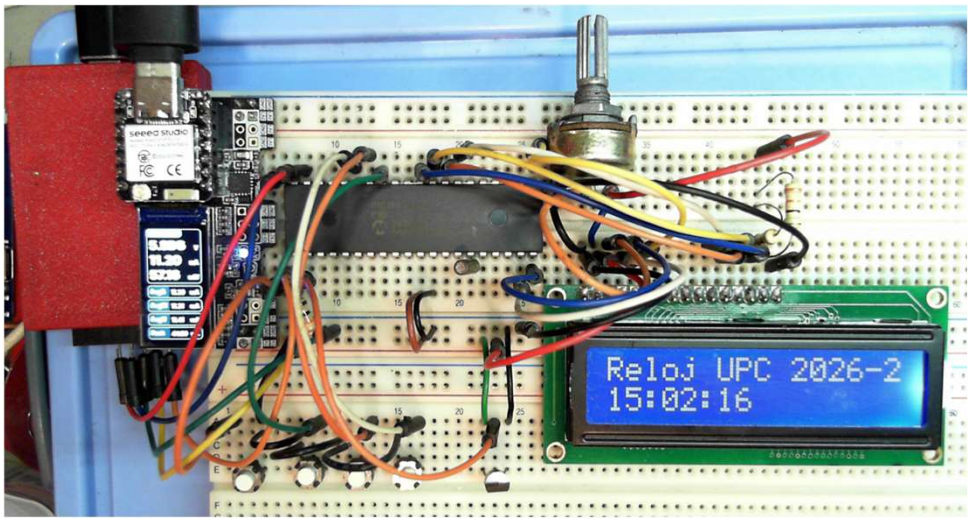
void main(void) {
    configuro();
    POS_CURSOR(1,0);
    ESCRIBE_MENSAJE2("Reloj UPC 202620");
    while(1){
        //aca vas a visualizar la hora actual
        POS_CURSOR(2,0);
        LCD_ESCRIBE_VAR_CHAR(horas,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(minutos,2);
        ENVIA_CHAR(':');
        LCD_ESCRIBE_VAR_CHAR(segundos,2);
    }
}

void __interrupt(high_priority) TMR1_ISR(void){
    PIR4bits.TMR1IF = 0;    //bajamos el flag TMR1IF;
    TMR1H = 0x80;
    TMR1L = 0x00;       //precarga de cuenta de 32768 a TMR1
    if(segundos == 59){
        segundos = 0;
        if(minutos == 59){
            minutos = 0;
            if(horas == 23){
                horas = 0;
            }
            else{
                horas++;
            }
        }
        else{
            minutos++;
        }
    }
    else{
        segundos++;
    }
}

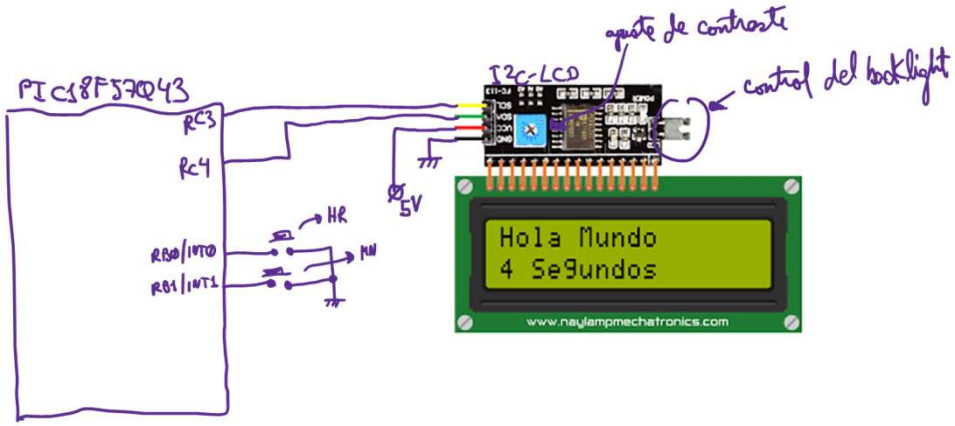
void __interrupt(low_priority) INTs_ISR(void){
    if(PIR0bits.INT1IF == 1){
        PIR0bits.INT1IF = 0;
        if(horas == 23){
            horas = 0;
        }
        else{
            horas++;
        }
    }
    else if(PIR0bits.INT2IF == 1){
        PIR0bits.INT2IF = 0;
        if(minutos == 59){
            minutos = 0;
        }
        else{
            minutos++;
        }
    }
}
```

Ejemplo 2026-20: Reloj empleando Timer1 y XTAL 32.768K

- Pruebas en físico



Mejora: empleo del modulo I2C-LCD

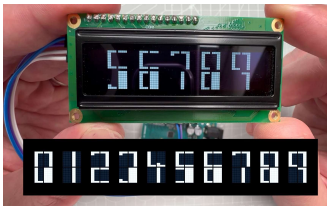


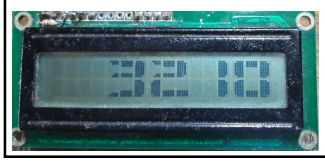
- Nota: Se debe de emplear la librería I2C_LCD (revisar el repositorio)

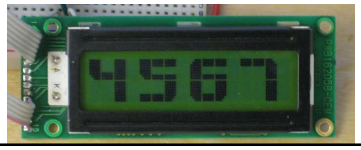
Ejercicio: Reloj empleando el Timer1



- Adicionales:
  - Cambio de formato 12H/24H
  - Sistema de alarma (activación/desactivación)
  - Función de cronómetro
  - Función de cuenta regresiva
  - Medición de temperatura / humedad con DHT11
  - Animación de secundero
  - Visualización de reloj analógico usando caracteres personalizados
  - Visualización de caracteres gigantes en el LCD
  - Sincronismo con NTP
  - Acelerómetro para encender la pantalla ante un movimiento

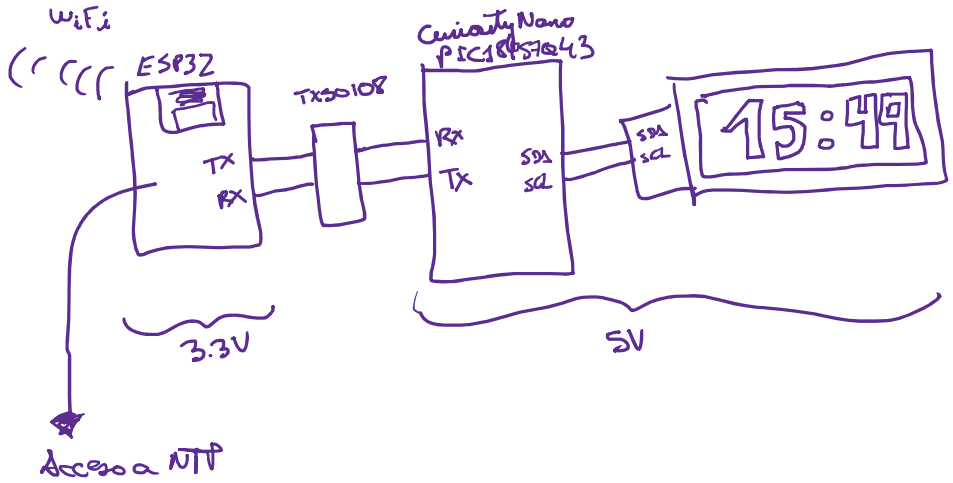








Adicional: Conectarlo el reloj a NTP



Fin de la sesión

- Destinar una hora el sábado y una hora el domingo para repasar el curso.